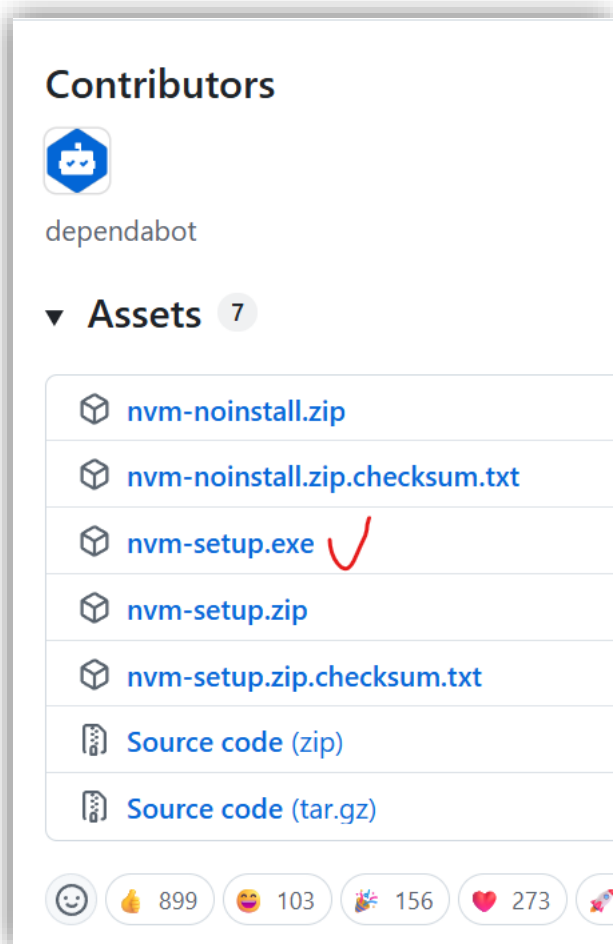


nvm 是專門用來控制和管理 Node.js 的小助手



<https://github.com/coreybutler/nvm-windows/releases>

查看版本號

```
PS C:\Users\Galaxy> nvm -v
1.2.2
PS C:\Users\Galaxy> node -v
v24.14.1
```

輸入nvm list查看電腦裡安裝 Node.js 版本的紀錄

```
PS C:\Users\Galaxy> nvm list

* 24.14.1 (Currently using 64-bit executable)
PS C:\Users\Galaxy> nvm install node
26.3.0
Downloading node.js version 26.3.0 (64-bit)...
Extracting node and npm...
Complete
Installation complete.
If you want to use this version, type:

nvm use 26.3.0
PS C:\Users\Galaxy> nvm use 26.3.0
Now using node v26.3.0 (64-bit)
```

創一個資料夾node-demo
cd到node-demo
然後輸入npm init -y

```
• PS C:\Users\Galaxy\Desktop\nodejs_course> cd node-demo
• PS C:\Users\Galaxy\Desktop\nodejs_course\node-demo> npm init -y
Wrote to C:\Users\Galaxy\Desktop\nodejs_course\node-demo\package.json:

{
  "name": "demo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

然後在app.js寫東西(例如"HELLO Molly")
輸入npm run dev
就會在終端機跑出寫的內容

然後在資料夾內創一個app.js
去腳本輸入這行：

```
node-demo > package.json > {} scripts > abc dev
1 {
2   "name": "demo",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "dev": "node app.js",
8     "test": "echo \"Error: no test specified\" && exit 1"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "type": "commonjs"
14 }
15
```

要改成module

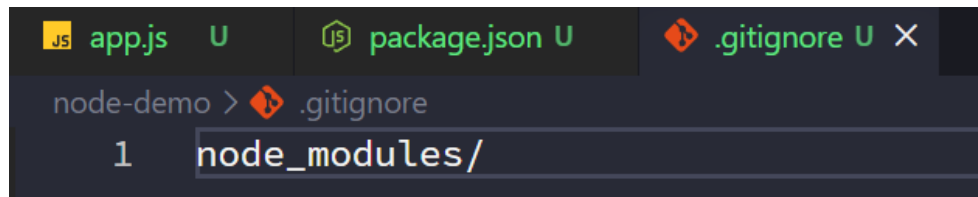
```
node-demo > app.js
1 console.log("HELLO Molly");
2

PS C:\Users\Galaxy\Desktop\nodejs_course\node-demo> npm init -y
"license": "ISC",
"type": "commonjs"
}

• PS C:\Users\Galaxy\Desktop\nodejs_course\node-demo> npm run dev
> demo@1.0.0 dev
> node app.js

HELLO Molly
PS C:\Users\Galaxy\Desktop\nodejs_course\node-demo>
```

先建立.gitignore檔案
寫這行防止等一下git 追蹤它
寫好再git init



The screenshot shows a code editor with three tabs: 'app.js', 'package.json', and '.gitignore'. The '.gitignore' tab is active, and the text 'node_modules/' is being entered on the first line. The editor's title bar indicates the current directory is 'node-demo'.

```
node-demo > .gitignore
1 node_modules/
```

```
// 做一個server
import http from "node:http";

const hostname = "localhost";

const port = 3000;

// req = request = 請求; res = response = 回應
const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain");
  res.end("Hello, Molly!\n");
});

server.listen(port, hostname, () => {
  console.log("執行中");
});
```

npm run dev

去瀏覽器搜尋<http://localhost:3000/>

但是每一次改內容就要每個步驟手動重啟很麻煩

解法一

安裝nodemon：

`npm i nodemon`

去修改腳本：

`"dev": "nodemon app.js",`

然後`npm run dev`

去瀏覽器開<http://localhost:3000/>

只要F5重新整理就可以順利更新了

解法二

去修改腳本：

`"dev": "node app.js --watch",`

然後`npm run dev`

去瀏覽器開<http://localhost:3000/>

只要F5重新整理就可以順利更新了

如何開始一個新的後端專案

1. 新增資料夾
2. vscode終端機Cd到新資料夾
3. npm init -y 產生package.json
4. 把package.json裡面的"type": "commonjs"改成"type": "module" 目的是要讓後續的開發可以用 import, export 這些現代化語法
5. 安裝 express : npm install express -save
6. 新增app.js
7. 寫內容
8. 在package.json腳本新增"dev": "node app.js",
9. npm run dev
10. 在瀏覽器搜尋<http://localhost:3000/todos>
11. 每一次改內容很麻煩=>下載nodemon
12. npm i nodemon
13. 去修改腳本 : "dev": "nodemon app.js",
14. npm run dev
15. 去瀏覽器開<http://localhost:3000/todos>
16. 只要F5重新整理就可以順利更新了
17. 把<http://localhost:3000/todos>貼到Postman看資料

```
▷ 偵錯
6   "scripts": {
7     "dev": "node app.js",
```

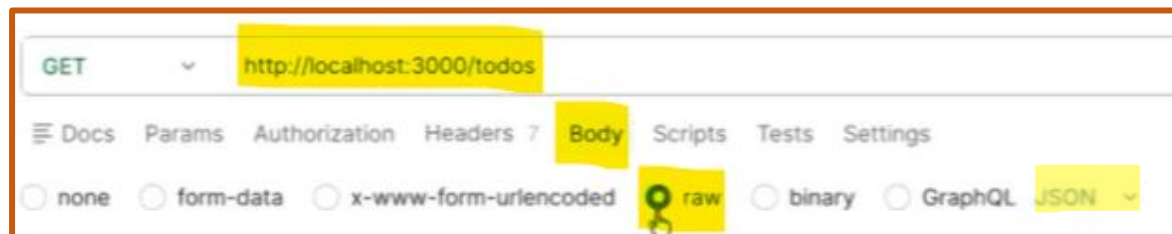
第七點寫內容：

```
import express from "express";

const app = express();

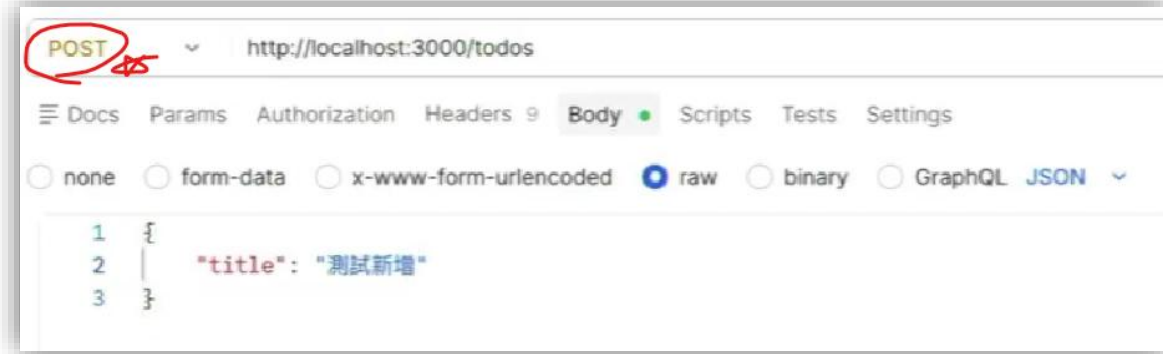
app.get("/todos", (req, res) => {
  res.send("eat sleep game movie");
});

app.listen(3000, () => {
  console.log("執行中");
});
```



```
// 示範在Postman新增資料搭配的寫法
import express from "express";
const app = express();
app.use(express.json());
const todos = [
  { id: 1, title: "吃飯" },
  { id: 2, title: "睡覺" },
];
app.post("/todos", (req, res) => {
  const { title } = req.body; // post傳遞資料給後端 通常不會由前端自由傳id 而是由後端來決定
  const newTodo = {
    id: todos.length + 1, // 目前有2筆資料等於長度2 新增資料所以要加一
    title,
  };
  todos.push(newTodo);
  res.status(201).json(todos);
});
app.get("/todos", (req, res) => {
  res.json(todos); // 把send改成json
});
app.listen(3000, () => {
  console.log("執行中");
});
```

然後新增一筆資料 左上角GET要改成POST
下面空白處自己寫入新資料





補上/123

那個123是自己想傳進去的東西(也就是這個示範中的id)

```
// 新增patch用法
app.patch("/todos/:id", (req, res) => {
  console.log(req.params.id);
  const id = parseInt(req.params.id); // parseInt可以換成
  Number
  const todo = todos.find((t) => t.id === id); // 從資料中找到相
  對應的id

  if (!todo) {
    return res.status(404).json({ error: "找不到這個todo" });
  }

  const { title } = req.body; // 這邊是前端帶入的要修改的資料

  if (title) {
    // 如果前面有帶入要修改的資料
    todo.title = title; // 將找到的 title 更新成帶入的 title
  }
  res.status(200).json(todos); // 看成功會回傳那些東西
});
```



補上/2
假設想刪除id:2的整筆資料

```
// 新增delete用法
app.delete("/todos/:id", (req, res) => {
  const id = Number(req.params.id);

  const ex = todos.some((t) => t.id === id); // some()不拿資料，只告訴我true/false

  todos = todos.filter((t) => t.id !== id);
  console.log(todos);
  res.status(204);
});

app.listen(3000, () => {
  console.log("執行中");
});
```


LOCAL | PostgreSQL

Items Queries History

Search for item...

- Functions
- Tables
 - customers
 - drinks
 - order_item_toppi...
 - order_items
 - orders
 - toppings

customers

```

1 SELECT name, price
2 FROM drinks
3 WHERE name LIKE '%珍珠%'
4 order by price
5 LIMIT 3

```

line 5, column 8, location 80

	name	price
1	珍珠紅茶	50
2	珍珠奶茶	55
3	珍珠奶綠	60

LOCAL | PostgreSQL 18.

Items Queries History

Search for item...

- Functions
- Tables
 - customers
 - drinks
 - order_item_toppi...
 - order_items
 - orders
 - toppings

customers

```

1 SELECT id, name, category, price
2 FROM drinks
3 WHERE category = '鮮奶'
4 order by price DESC
5 LIMIT 3

```

line 1, column 12, location 11

	id	name	category	price
1	4	黑糖珍珠鮮奶	鮮奶	75
2	14	抹茶拿鐵	鮮奶	70
3	13	鮮奶茶	鮮奶	60

```

1 SELECT * from drinks;
2 INSERT INTO drinks (name, category, price)
3 VALUES ('草莓奶茶', '奶茶', 65);
4

```

```

1 UPDATE drinks
2 set price = 60
3 where name = '珍珠奶茶'
4 SELECT * FROM drinks WHERE name LIKE '%奶茶%';|

```

```

1 DELETE FROM drinks
2 WHERE name = '綠茶'|

```

```

1 SELECT AVG(price) FROM drinks

```

line 1, column 30, location 29

	avg
1	59.000000000000000000

```

1 SELECT category, count(*) AS drinks
2 FROM drinks
3 group by category

```

line 3, column 19, location 68

	category	drinks
1	奶茶	5
2	果汁	4
3	純茶	3
4	鮮奶	3

```

1 SELECT category, count(*) AS drinks
2 FROM drinks
3 WHERE category IN ('奶茶', '純茶', '鮮奶', '果汁')
4 GROUP BY category;|

```

line 4, column 19, location 112

	category	drinks
1	純茶	3
2	鮮奶	3
3	奶茶	5
4	果汁	4

1	SELECT	category,	ROUND(AVG(price))	AS	avg_price
2	FROM	drinks			
3	GROUP BY	category			
4	ORDER BY	avg_price	DESC;		

line 4, column 25, location 105

	avg_price	category	
1	72	鮮奶	
2	65	奶茶	
3	54	果汁	
4	43	純茶	

AVG(price)：計算該類別所有飲料的 price (價格) 平均值

ROUND(...)：將計算出來的平均價格進行四捨五入取到整數

AS avg_price：將這個計算結果欄位取一個好讀的別名叫作 avg_price

GROUP BY category：將資料依照 category 進行分組

ORDER BY avg_price DESC：將最後算出來的結果依照 avg_price (平均價格) 進行DESC 降冪排序 (從高到低)

1	SELECT	drink_id,	sum(qty)	as	sold
2	FROM	order_items			
3	GROUP BY	drink_id;			

line 3, column 19, location 71

	drink_id	sold	
1	11 →	1	
2	9 →	2	
3	3 →	3	
4	5 →	1	
5	4 →	3	
6	10 →	1	
7	6 →	3	
8	14 →	1	
9	13 →	2	
10	2 →	1	
11	12 →	1	
12	1 →	11	
13	8 →	3	

SUM(qty)：將該款飲料在所有訂單中的 qty (銷售數量) 全部加總起來

AS sold：將加總後的數量欄位取一個好讀的別名叫做 sold (已售出數量)

```

1 SELECT customer_id, count(*) as orders
2 FROM orders
3 GROUP BY customer_id
4 order by orders desc;

```

line 4, column 22, location 96

	customer_id	orders	
1	1 →	3	
2	3 →	2	
3	5 →	2	
4	4 →	2	
5	2 →	2	
6	6 →	1	

COUNT(*)：負責計算每個顧客在資料表裡
總共出現了幾筆紀錄

ORDER BY orders DESC：orders是第一
行用 AS orders 幫 COUNT(*) 取的別名

```

1 SELECT d.name, oi.qty
2 from order_items oi
3 join drinks d on d.id = oi.drink_id;

```

line 2, column 20, location 42

	name	qty	
1	珍珠奶茶	2	
2	紅茶	1	
3	珍珠奶茶	1	
4	珍珠奶綠	1	
5	黑糖珍珠鮮奶	2	
6	珍珠奶茶	1	
7	烏龍奶茶	2	
8	珍珠奶茶	3	
9	檸檬綠茶	1	
10	珍珠奶茶(大)	1	
11	抹茶拿鐵	1	
12	珍珠奶茶	1	
13	鮮奶茶	2	
14	珍珠奶綠	2	
15	珍珠紅茶	1	
16	珍珠奶茶	2	
17	冬瓜檸檬	1	
18	黑糖珍珠鮮奶	1	
19	冬瓜茶	2	
20	烏龍奶茶	1	
21	珍珠奶茶	1	
22	葡萄柚綠茶	1	
23	紅茶	2	

找出所有被點過的飲料名稱以及它們被點
購的數量

SELECT d.name, oi.qty：想看兩樣資料，
飲料名稱（name）和訂購的數量（qty）

FROM order_items oi：撈取的資料庫表
格叫「訂單明細（order_items）」，並
在後面給它一個簡短代號 oi

JOIN drinks d ON d.id = oi.drink_id：把
「飲料清單（drinks）」這個表格串聯進
來「訂單明細（order_items）」表格旁
邊，並給飲料清單（drinks）一個代號 d。
串聯的依據是兩邊都有的「飲料編號（id
與 drink_id）」

```
1 SELECT b.name, a.qty, b.price
2 from order_items a
3 JOIN drinks b on b.id = a.drink_id;
```

在 SQL 查詢語句中，a 和 b 被稱為資料表別名 (Table Aliases) 。
第三行 on 是指依照接下來的「條件」對齊資料。

```
1 SELECT c.*, o.*, oi.*, d.*
2 FROM customers c
3 JOIN orders o ON o.customer_id = c.id
4 JOIN order_items oi ON oi.order_id = o.id
5 JOIN drinks d ON d.id = oi.drink_id;
```

用JOIN可以不斷關聯下去

```
1 SELECT d.name, SUM(oi.qty)
2 FROM order_items oi
3 JOIN drinks d ON d.id = oi.drink_id
4 GROUP BY d.name
5 ORDER BY SUM(oi.qty) DESC;
```

line 4, column 16, location 101

	name	sum
1	珍珠奶茶	11
2	烏龍奶茶	3
3	珍珠奶綠	3
4	紅茶	3
5	黑糖珍珠鮮奶	3
6	冬瓜茶	2
7	鮮奶茶	2
8	冬瓜檸檬	1
9	葡萄柚綠茶	1
10	珍珠紅茶	1
11	抹茶拿鐵	1
12	檸檬綠茶	1
13	珍珠奶茶(大)	1

ORDER BY：請排序

```
1 SELECT d.name, oi.qty
2 FROM order_items oi
3 JOIN drinks d ON d.id = oi.drink_id
4 WHERE oi.order_id = 6;
```

line 4, column 22, location 102

	name	qty
1	珍珠奶茶(大)	1
2	抹茶拿鐵	1
3	珍珠奶茶	1

找出訂單第六筆資料

```

1 SELECT c.name, SUM(d.price * oi.qty) as total
2 FROM customers c
3 JOIN orders o ON o.customer_id = c.id
4 JOIN order_items oi ON oi.order_id = o.id
5 JOIN drinks d ON d.id = oi.drink_id
6 GROUP BY c.name
7 ORDER BY total DESC;

```

line 7, column 21, location 221

	name	total
1	小明	415
2	志明	400
3	阿美	395
4	春嬌	335
5	小華	300
6	大雄	130

JOIN orders o ON o.customer_id = c.id

先查出「這個客戶 (c.id) 到底下了哪幾筆訂單 (o.customer_id)」。

JOIN order_items oi ON oi.order_id = o.id

用訂單編號 (o.id) 去對齊點單明細 (oi.order_id) 。這時終於知道這筆訂單裡「買了幾杯 (qty)」以及「買了哪幾款飲料的編號 (drink_id)」。

JOIN drinks d ON d.id = oi.drink_id

拿明細裡的飲料編號 (oi.drink_id) ，去對齊飲料菜單的編號 (d.id) 。這時候我們終於查到了這款飲料的「名字」和「單價 (d.price)」。

```
1 SELECT t.name, COUNT(*)
2 FROM order_item_toppings oit
3 JOIN toppings t ON t.id = oit.topping_id
4 GROUP BY t.name;
```

line 4, column 17, location 113

	name	count	
1	愛玉	1	
2	珍珠	10	
3	仙草凍	1	
4	布丁	3	
5	波霸	1	
6	椰果	3	

COUNT(*)：數每個配料總共被點了幾次。

FROM order_item_toppings oit：以明細表(記錄了每一次加料)為主角。

JOIN toppings t ON ...：連上配料菜單，把配料的編號換成「真實名字」。

GROUP BY t.name：把相同名字的配料打包成一堆，好讓COUNT(*)去算每一堆的總數。

```

1 SELECT *
2 FROM toppings t
3 LEFT JOIN order_item_toppings oit
4 ON oit.topping_id = t.id;

```

line 4, column 25, location 86

	id	name	price	order_item_id	topping_id
1	1	珍珠	10	1 →	1 →
2	3	布丁	15	1 →	3 →
3	1	珍珠	10	3 →	1 →
4	1	珍珠	10	5 →	1 →
5	4	仙草凍	10	5 →	4 →
6	1	珍珠	10	6 →	1 →
7	2	椰果	10	6 →	2 →
8	2	椰果	10	7 →	2 →
9	6	愛玉	10	7 →	6 →
10	1	珍珠	10	8 →	1 →
11	1	珍珠	10	10 →	1 →
12	7	波霸	10	10 →	7 →
13	1	珍珠	10	12 →	1 →
14	3	布丁	15	13 →	3 →
15	2	椰果	10	14 →	2 →
16	1	珍珠	10	16 →	1 →
17	3	布丁	15	16 →	3 →
18	1	珍珠	10	20 →	1 →
19	1	珍珠	10	21 →	1 →
20	8	燕麥	15	NULL →	NULL →
21	5	蘆薈	15	NULL →	NULL →

列出所有的配料 (toppings) 並把每款配料被點過的明細 (oit) 拼在右邊。

LEFT JOIN：不管有沒有人點過，所有配料一律都會顯示出來。

```

1 SELECT *
2 FROM toppings t
3 LEFT JOIN order_item_toppings oit
4 ON oit.topping_id = t.id
5 WHERE oit.topping_id IS null;

```

line 5, column 30, location 117

	id	name	price	order_item_id	topping_id
1	8	燕麥	15	NULL →	NULL →
2	5	蘆薈	15	NULL →	NULL →

只顯示從來沒有人點過的配料(找沒人氣)

```

1 SELECT t.name, COUNT(*) AS times
2 FROM order_item_toppings oit
3 JOIN toppings t ON oit.topping_id = t.id
4 GROUP BY t.name
5 ORDER BY times DESC
6 LIMIT 3;

```

line 6, column... No limit Beautify Run

	name	times	
1	珍珠	10	
2	布丁	3	
3	椰果	3	

```

1 SELECT DISTINCT d.name
2 FROM order_item_toppings oit
3 JOIN toppings t ON t.id = oit.topping_id
4 JOIN order_items oi ON oi.id = oit.order_item_id
5 JOIN drinks d ON d.id = oi.order_id
6 WHERE t.name = '珍珠';

```

line 3, column... No limit Beautify Run Current

	name	
1	冬瓜茶	
2	珍珠奶茶	
3	珍珠奶茶(大)	
4	珍珠奶綠	
5	珍珠紅茶	
6	紅茶	
7	檸檬綠茶	

找出所有加過「珍珠」的飲料名稱。

DISTINCT是去除重複。

JOIN order_items oi ON oi.id = oit.order_item_id：連到「點過的單品清單」，知道這一份配料是加在某一杯特定的飲料裡。


```
1 SELECT SUM(t.price)
2 FROM order_item_toppings oit
3 JOIN toppings t ON t.id = oit.topping_id
```

line 3, column... No limit Beautify Run C

	sum
1	205

計算所有訂單中，賣出的配料總共賺了多少錢

```
1 SELECT t.name
2 from toppings t
3 LEFT JOIN order_item_toppings oit
4 ON t.id = oit.topping_id
5 WHERE oit.topping_id IS NULL;
```

line 5, column... No limit Beautify

	name
1	燕麥
2	蘆薈

找出沒有被點過的配料

如何用 Express 建立後端伺服器

1. 建立資料夾 終端機輸入`mkdir` 資料夾名
2. 打開該資料夾的`vscode`
3. `Npm init -y` 生成`package.json`
4. `Type`後面改`module`
5. `Npm install express dotenv pg` 安裝這三個東西
6. 新增`app.js`
7. 去`package.json`寫腳本`"dev": "node app.js",`
8. 新增 `.env` 檔案

postgresql://postgres:password@localhost:5432/bubbletea

- **postgresql://**：協定名稱（告訴電腦要用 PostgreSQL 連線）。
- **postgres**：名稱（資料庫的使用者帳號）。
- **password**：密碼（該帳號的密碼）。
- **localhost:5432**：主機 + port（localhost 代表你自己的電腦，5432 是 PostgreSQL 預設的通訊埠）。
- **bubbletea**：資料庫名

```
.env
1 DATABASE_URL=postgresql://postgres:anit01316@localhost:5432/
  bubbletea
2 API_URL=
3
```

我PostgreSQL的密碼

```
app.js > ...
1 import "dotenv/config";
2 import express from "express";
3 import { Pool } from "pg";
4
5 const pool = new Pool({
6   connectionString: process.env.DATABASE_URL,
7 });
8
9 const app = express();
10
11 app.get("/menu", async (req, res) => {
12   const { rows } = await pool.query("SELECT name, category,
13     price FROM drinks");
14   res.json(rows);
15 });
16
17 app.listen(3000, () => {
18   console.log("執行中");
19 });
```

然後 npm run dev
然後去瀏覽器輸入 <http://localhost:3000/menu>

Drizzle orm

1. 建立新的專案

- 新增資料夾
- `npm init -y`
- 把 `type` 改成 `"module"`

2. 安裝套件

```
npm install drizzle-orm pg dotenv
```

```
npm install -D drizzle-kit
```

3. 設定連線資料庫

- 新增 `.env` 到根目錄
- 給一個變數名稱（通常是大寫常數），例如 `DATABASE_URL`
- 連線位置設定，例
如：`DATABASE_URL=postgresql://postgre:password@localhost:5432/bubbletea_orm`
- 通常 `.env` 環境變數不會加入版控，所以會寫到 `.gitignore` 中

```
12   "type": "module",
13   "dependencies": {
14     "dotenv": "^17.4.2",
15     "drizzle-orm": "^0.45.2",
16     "pg": "^8.21.0"
17   },
18   "devDependencies": {
19     "drizzle-kit": "^0.31.10"
20   }
21 }
```

```
.env
drizzle.config.js
.gitignore
.gitignore
1  .env
2  node_modules/
```

4. 設定 drizzle.config.js

drizzle.config.js

- 在根目錄新增一個 drizzle.config.js

```
import 'dotenv/config'
import { defineConfig } from 'drizzle-kit'

export default defineConfig({
  out: './drizzle',
  schema: './src/db/schema.js',
  dialect: 'postgresql',
  dbCredentials: { url: process.env.DATABASE_URL }
})
```

5. 定義 drinks 表

schema.js

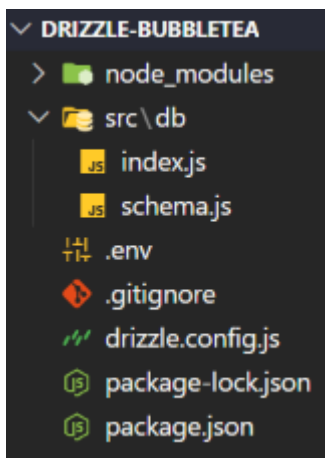
```
import { pgTable, integer, varchar } from 'drizzle-orm/pg-core'

export const drinks = pgTable('drinks', {
  id: integer().generatedAlwaysAsIdentity().primaryKey(),
  name: varchar({ length: 100 }).notNull(),
  category: varchar({ length: 20 }).notNull(),
  price: integer().notNull()
})
```

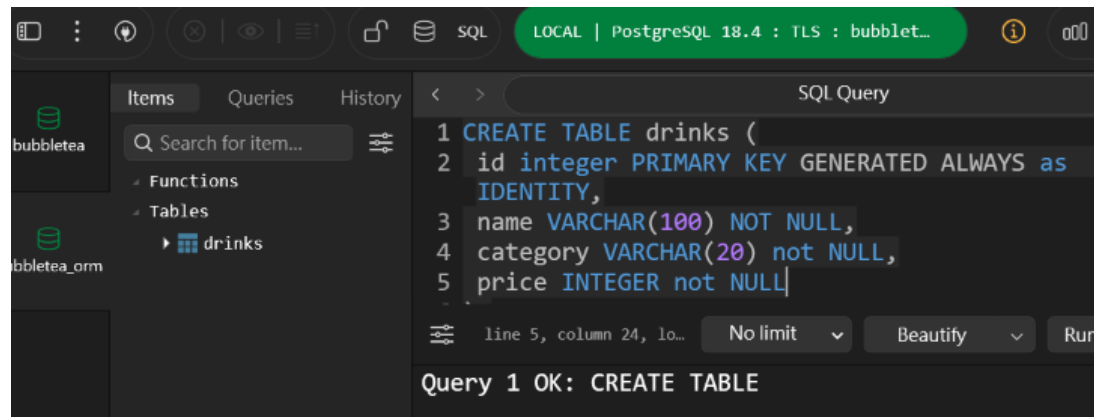
比照 SQL 原生語法：

```
CREATE TABLE drinks (
  id INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name VARCHAR(100) NOT NULL,
  category VARCHAR(20) NOT NULL,
  price INTEGER NOT NULL
);
```

在 schema.js 定義表格搭配在 package.json 新增 scripts 加入 “push” 和 “studio” 的快速鍵以及執行 npm run push 來創建 drinks 表



直接在TablePlus寫原生SQL語法創建drinks表的方法



6. 建立 DB 連線跟連線池

放在 `src/db/index.js` `index.js`

```
import { drizzle } from 'drizzle-orm/node-postgres'
import pg from 'pg'
import * as schema from './schema.js'

const { Pool } = pg

const pool = new Pool({
  connectionString: process.env.DATABASE_URL
})

export const db = drizzle(pool, { schema })
```

第六步驟可以放在第七步驟後面
因為創建drinks表的步驟是做完步驟五然後做步驟七

7. 按照規劃新增一個表

在 `package.json` 的 `scripts` 新增：

```
"push": "drizzle-kit push",
"studio": "drizzle-kit studio",
```

執行 `npm run push` 就會按照規劃的 `schema` 新增 `table` 在資料庫中

```
6   "scripts": {
7     "push": "drizzle-kit push",
8     "studio": "drizzle-kit studio",
9     "test": "echo \"Error: no test specified\" && exit 1"
```

```
PS C:\Users\Galaxy\Desktop\drizzle-bubbletea> npm run push
>>
> drizzle-bubbletea@1.0.0 push
> drizzle-kit push

No config path provided, using default 'drizzle.config.js'
Reading config file 'C:\Users\Galaxy\Desktop\drizzle-bubbletea\drizzle.config.js'
Using 'pg' driver for database querying
[✓] Pulling schema from database...
[✓] Changes applied
```

看到兩個打勾就是push成功
去TablePlus按重新整理

如果想繼續新增表或更新，就在`schema.js`編輯好再`npm run push`就好

src > db >  schema.js > ...

```
1  import {
2    pgTable,
3    integer,
4    varchar,
5    text,
6    timestamp,
7  } from "drizzle-orm/pg-core";
8
9  // integer 必為整數; generatedAlwaysAsIdentity 自動生成ID; primaryKey 全表唯一的身分證，
   // 絕對不能重複，也不能不填; varchar 可以變動長度的文字(告訴資料庫，這一欄是要用來存「中文字、英文、
   // 數字或符號」的); notNull 表示不能為空值，也就是要求必填
10
11  export const drinks = pgTable("drinks", {
12    id: integer().generatedAlwaysAsIdentity().primaryKey(),
13    name: varchar({ length: 100 }).notNull(),
14    category: varchar({ length: 20 }).notNull(),
15    price: integer().notNull(),
16  });
17
18  export const toppings = pgTable("toppings", {
19    id: integer().generatedAlwaysAsIdentity().primaryKey(),
20    name: varchar({ length: 50 }).notNull(),
21    price: integer().notNull(),
22  });
23
24  export const customers = pgTable("customers", {
25    id: integer().generatedAlwaysAsIdentity().primaryKey(),
26    name: varchar({ length: 50 }).notNull(),
27    phone: varchar({ length: 30 }),
28  });
```



```
30
31 // references 建立外鍵 (Foreign Key) 限制，強制規定這個格子裡填寫的客戶編號必須真的存在於 customers 表格的
    id 欄位中; timestamp 記錄「日期 + 時間」(包含時分秒); { withTimezone: true } 負責時區功能; defaultNow 預
    設值為現在
32 // { onDelete: "set null" } 功能：當客戶資料被刪除時，自動把該客戶的所有訂單的「客戶編號」格子變空白 (Null)，
    但不會刪除訂單本身
33 export const orders = pgTable("orders", {
34   id: integer().generatedAlwaysAsIdentity().primaryKey(),
35   customerId: integer("customer_id").references(() => customers.id, {
36     onDelete: "set null",
37   }),
38   createdAt: timestamp("created_at", { withTimezone: true })
39     .notNull()
40     .defaultNow(),
41 });
42
43 // { onDelete: "cascade" } 功能：如果源頭母資料被刪除，那綁在底下的所有子資料也會自動連帶刪除
44 // { onDelete: "restrict" } 功能：嚴格禁止刪除
45 // default(1) 預設數量至少要有1
46 export const orderItems = pgTable("order_items", {
47   id: integer().generatedAlwaysAsIdentity().primaryKey(),
48   orderId: integer("order_id")
49     .references(() => orders.id, { onDelete: "cascade" })
50     .notNull(),
51   drinkId: integer("drink_id")
52     .references(() => drinks.id, { onDelete: "restrict" })
53     .notNull(),
54   qty: integer().notNull().default(1),
55 });
56
```

```
57 // (t) => [primaryKey({ columns: [t.orderItemId, t.toppingId] })] 功能：把 orderItemId 和
    toppingId 綁在一起，規定這兩個數字的組合在整張表裡只能出現一次，防止重複加料
58 export const orderItemToppings = pgTable(
59   "order_item_toppings",
60   {
61     orderItemId: integer("order_item_id")
62       .notNull()
63       .references(() => orderItems.id, { onDelete: "cascade" }),
64     toppingId: integer("topping_id")
65       .notNull()
66       .references(() => toppings.id, { onDelete: "restrict" }),
67   },
68   (t) => [primaryKey({ columns: [t.orderItemId, t.toppingId] })],
69 );
```

資料庫初始化與種子數據灌入

「Seeding (灌種子/播種) 」是一個專有名詞。

指的是在開發或測試階段，自動往空白的資料庫裡塞入一堆預先準備好的模擬數據 (測試資料) 。

建立src/db/seed.js

寫入老師提供的種子資料，也可以叫AI生

去package.json寫腳本"seed": "node src/db/seed.js",

執行 npm run seed

成功：

```
PS C:\Users\Galaxy\Desktop\drizzle-bubbletea> npm run seed

> drizzle-bubbletea@1.0.0 seed
> node src/db/seed.js

■種子資料灌完：
  飲料 15、加料 8、顧客 6
  訂單 12、明細 23、加料記錄 19
PS C:\Users\Galaxy\Desktop\drizzle-bubbletea> 
```

建立src/db/app.js

寫內容

去package.json寫腳本"dev": "node src/db/app.js",

執行 npm run dev

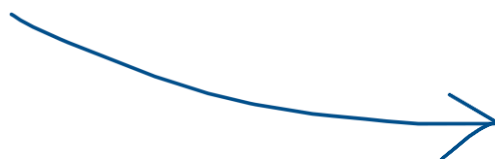
成功：

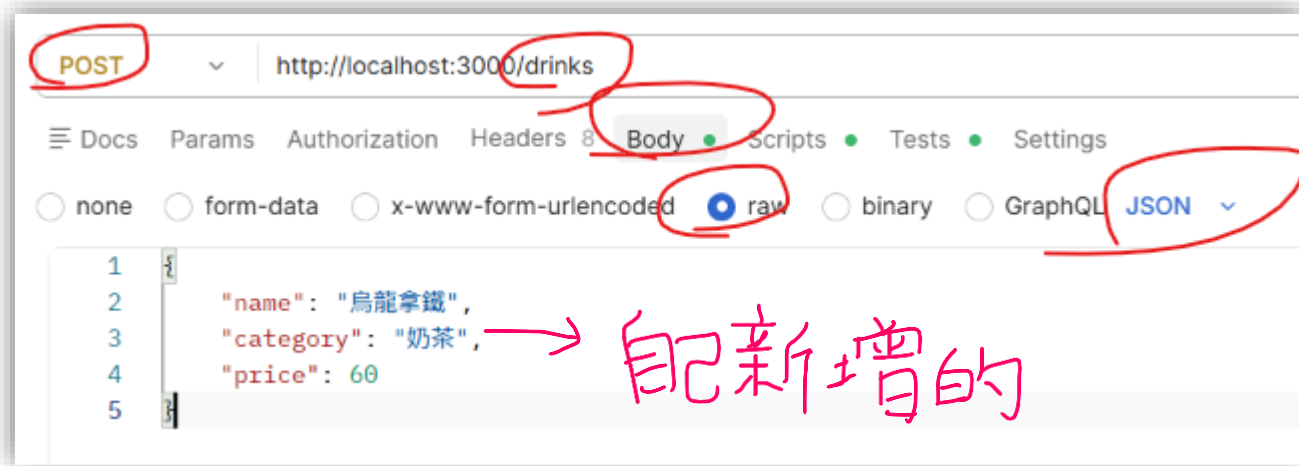
```
PS C:\Users\Galaxy\Desktop\drizzle-bubbletea> npm run dev

> drizzle-bubbletea@1.0.0 dev
> node src/db/app.js

執行中 http://localhost:3000
```

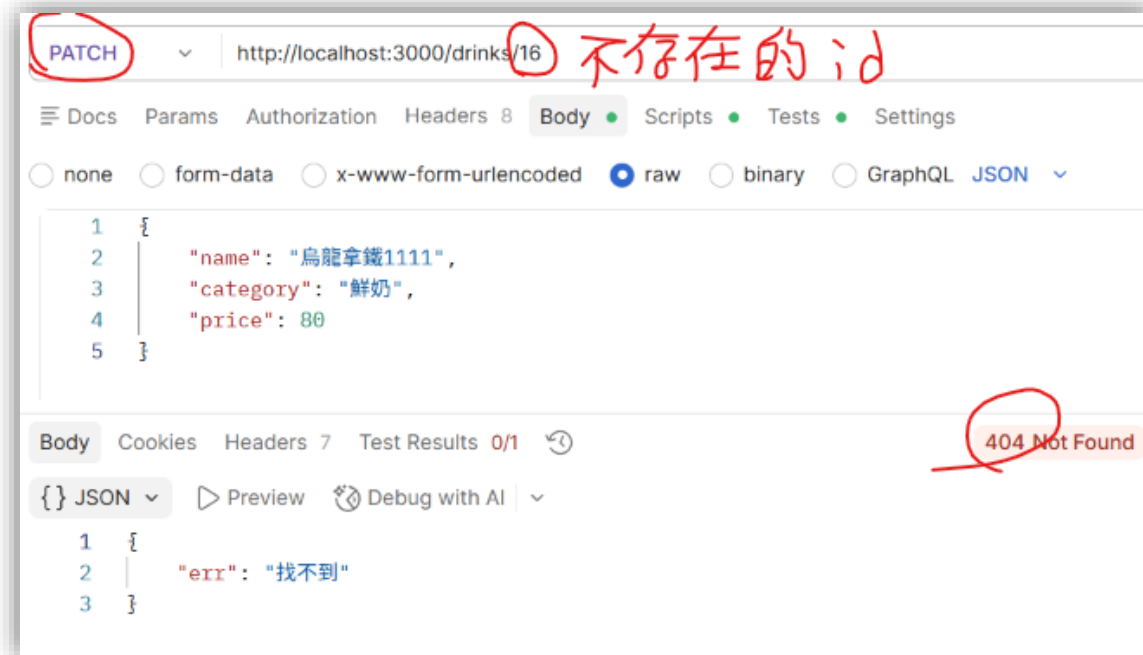
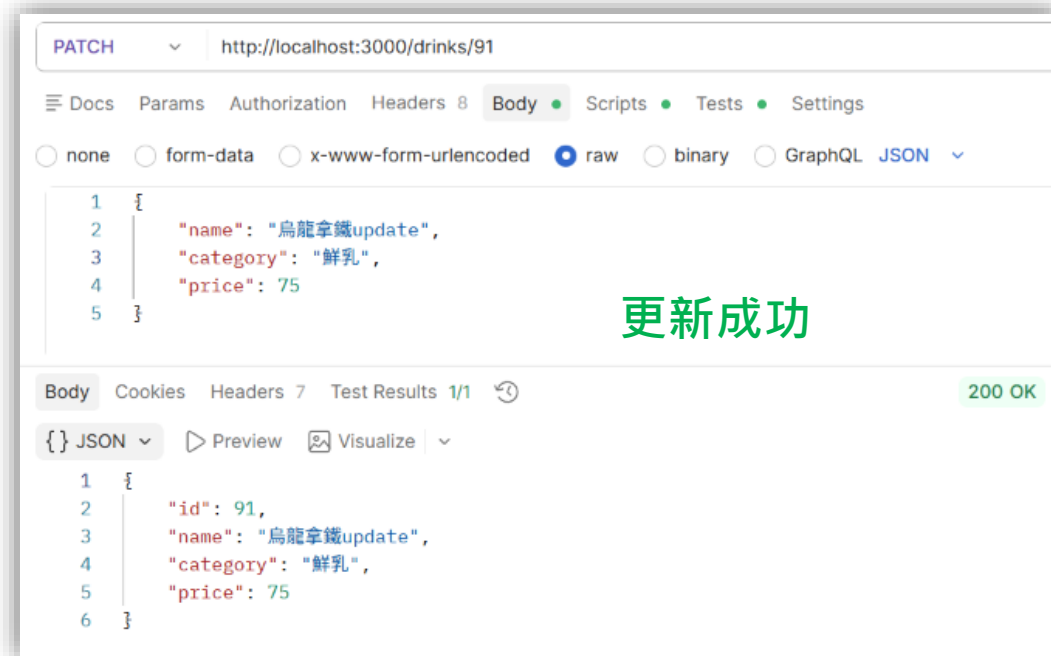
去瀏覽器開 <http://localhost:3000/drinks>



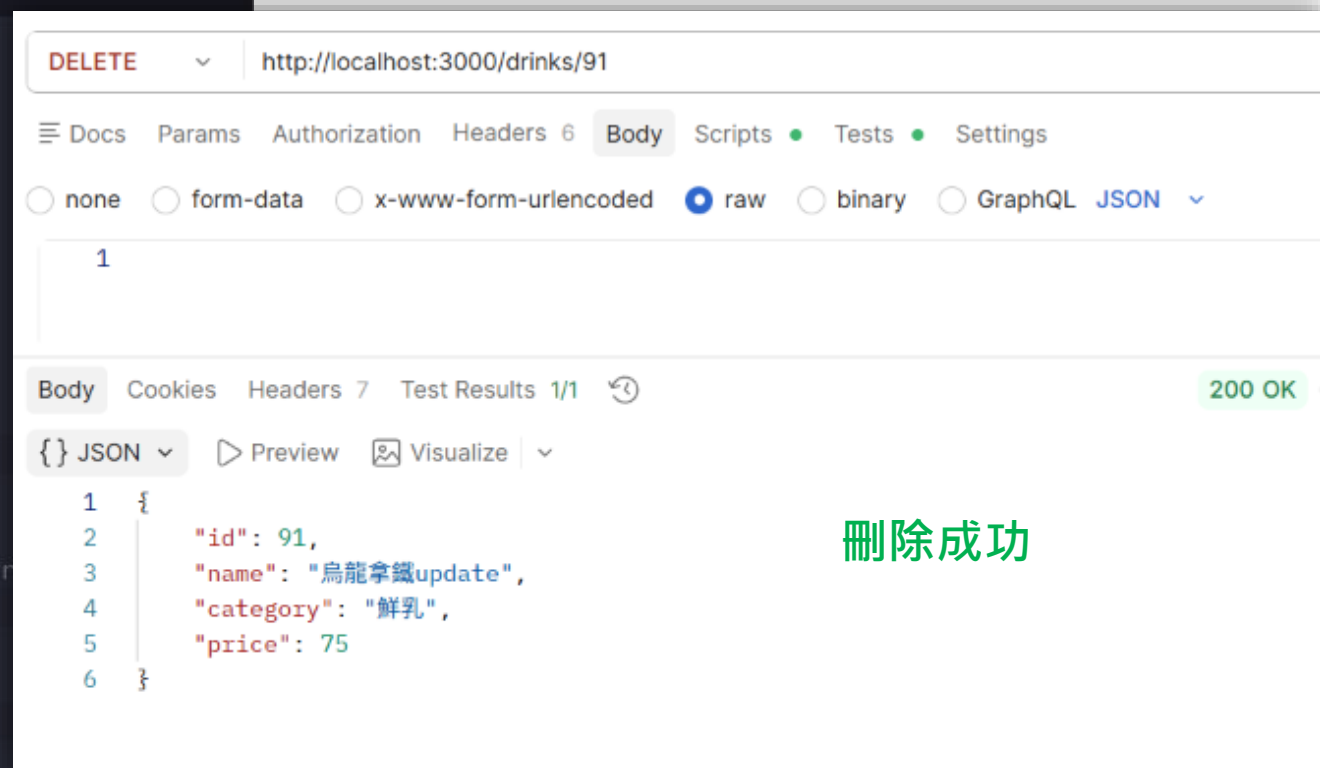


```
app.js schema.js index.js
src > db > app.js > ...
1 import "dotenv/config";
2 import express from "express";
3 import { db } from "../index.js";
4 import { drinks } from "../schema.js";
5 import { desc } from "drizzle-orm";
6
7 const app = express();
8
9 app.use(express.json());
10
11 app.get("/menu", async (req, res) => {
12   const rows = await db
13     .select({ name: drinks.name, price: drinks.price })
14     .from(drinks)
15     .orderBy(desc(drinks.price));
16   res.json(rows);
17 });
18
19 app.post("/drinks", async (req, res) => {
20   const { name, category, price } = req.body;
21   const [newDrinks] = await db
22     .insert(drinks)
23     .values({ name, category, price })
24     .returning();
25
26   res.status(201).json(newDrinks);
27 });
28
29 app.listen(3000, () => {
30   console.log("執行中 http://localhost:3000");
31 });
```

```
app.js
src > db > app.js > ...
1 import "dotenv/config";
2 import express from "express";
3 import { db } from "./index.js";
4 import { drinks } from "./schema.js";
5 import { desc, eq } from "drizzle-orm";
6
7 const app = express();
8
9 app.use(express.json());
10
11 > app.get("/menu", async (req, res) => { ...
17 });
18
19 // 使用 const [newDrinks] (陣列) 是因為 Drizzle ORM 的 .returning() 功能
20 > app.post("/drinks", async (req, res) => { ...
28 });
29
30 app.patch("/drinks/:id", async (req, res) => {
31   const id = req.params.id;
32   const { name, category, price } = req.body;
33
34   const [updateDrink] = await db
35     .update(drinks)
36     .set({ name, category, price })
37     .where(eq(drinks.id, id))
38     .returning();
39
40   if (!updateDrink) return res.status(404).json({ err: "找不到" });
41
42   res.json(updateDrink);
43 });
```



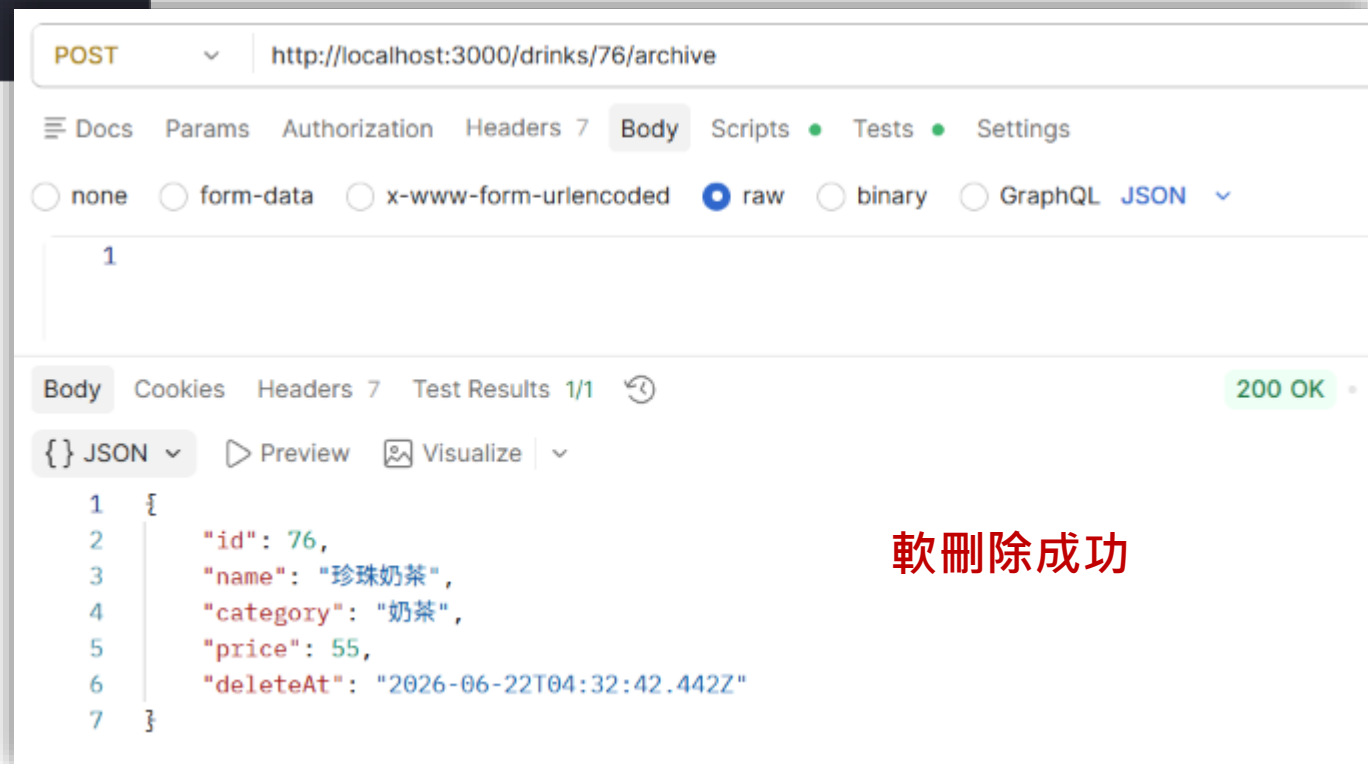

```
app.js
src > db > app.js > ...
1 import "dotenv/config";
2 import express from "express";
3 import { db } from "./index.js";
4 import { drinks } from "./schema.js";
5 import { desc, eq } from "drizzle-orm";
6
7 const app = express();
8
9 app.use(express.json());
10
11 > app.get("/menu", async (req, res) => {...
17 });
18
19 // 使用 const [newDrinks] (陣列) 是因為 Drizzle ORM 的 .return
20 > app.post("/drinks", async (req, res) => {...
28 });
29
30 > app.patch("/drinks/:id", async (req, res) => {...
43 });
44
45 app.delete("/drinks/:id", async (req, res) => {
46   const id = req.params.id;
47   const [deleteItem] = await db
48     .delete(drinks)
49     .where(eq(drinks.id, id))
50     .returning();
51   if (!deleteItem) return res.status(404).json({ err: "找不到" });
52
53   res.json(deleteItem);
54 });
55
```



```
88 // 查看一筆資料
89 app.get("/drinks/:id", async (req, res) => {
90   const id = req.params.id;
91   const [row] = await db
92     .select()
93     .from(drinks)
94     .where(eq(drinks.id, id))
95     .limit(1);
96
97   if (!row) return res.status(404).json({ err: "找不到" });
98   res.json(row);
99 });
```

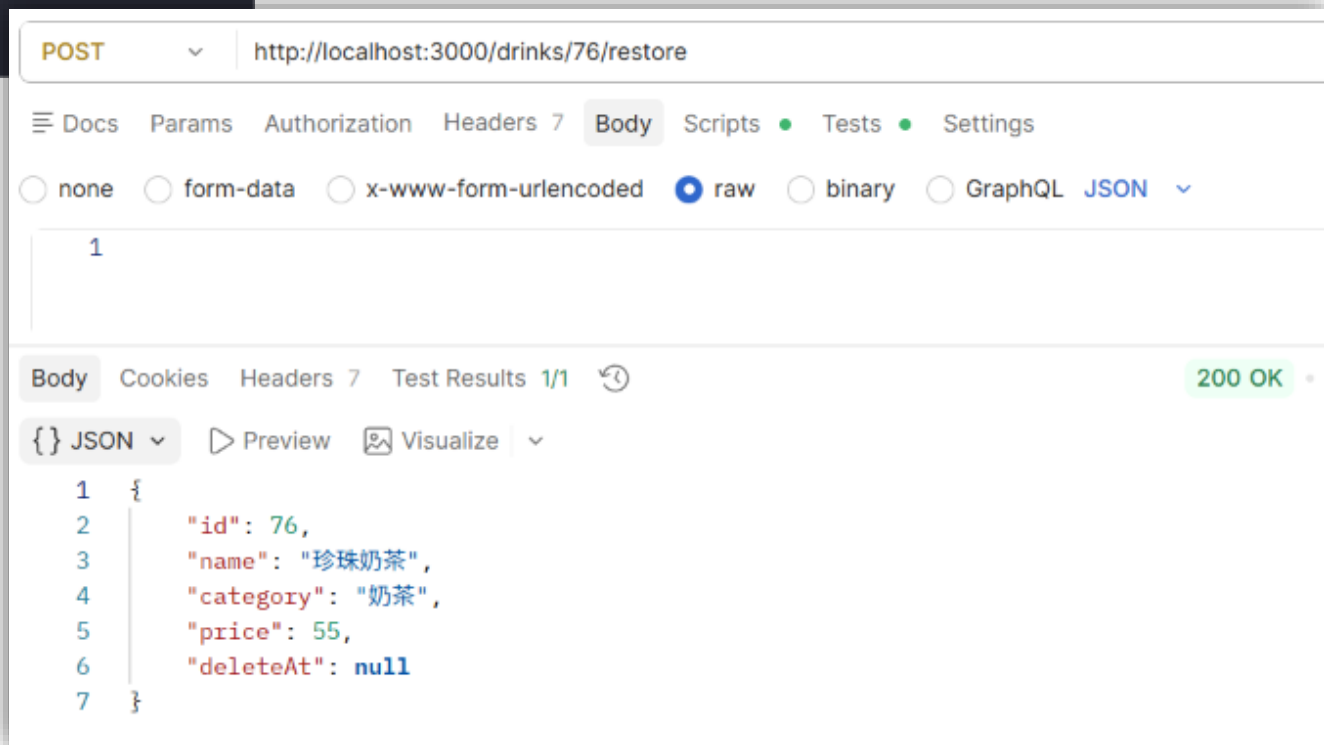


```
60 // 軟刪除一筆資料
61 app.post("/drinks/:id/archive", async (req, res) => {
62   const id = req.params.id;
63   const [archive] = await db
64     .update(drinks)
65     .set({ deleteAt: new Date() })
66     .where(eq(drinks.id, id))
67     .returning();
68
69   if (!archive) return res.status(404).json({ err: "找不到" });
70   res.json(archive);
71 });
72
```



```
src > db > app.js > ...
1  import "dotenv/config";
2  import express from "express";
3  import { db } from "./index.js";
4  import { drinks } from "./schema.js";
5  import { desc, eq, isNull } from "drizzle-orm";
6
7  const app = express();
8
9  app.use(express.json());
10
11 // 查看全部資料的飲料名稱和價格，並排除軟刪除的資料
12 app.get("/menu", async (req, res) => {
13   const rows = await db
14     .select({ name: drinks.name, price: drinks.price })
15     .from(drinks)
16     .where(isNull(drinks.deleteAt))
17     .orderBy(desc(drinks.price));
18
19   res.json(rows);
20 });
```

```
75 // 還原軟刪除的資料
76 app.post("/drinks/:id/restore", async (req, res) => {
77   const id = req.params.id;
78   const [item] = await db
79     .update(drinks)
80     .set({ deleteAt: null })
81     .where(eq(drinks.id, id))
82     .returning();
83
84   if (!item) return res.status(404).json({ err: "找不到" });
85   res.json(item);
86 });
```



多人協作不會使用前面學的push，而是generate和migrate

在package.json寫腳本：

```
"generate": "drizzle-kit generate",
```

```
"migrate": "drizzle-kit migrate"
```

執行 **npm run generate** 產生 **SQL 遷移檔案**

- 它做了什麼：Drizzle 會去讀取在 TypeScript 程式碼裡寫的資料表結構（Schema，例如 orders、drinks 表），並和目前的設計圖做比對。
- 產出的結果：它會在專案裡自動產生一個 .sql 檔案（例如0000_sudden_omega_flight.sql）。
- 重要特點：這個步驟完全不會動到資料庫。它只是把 TypeScript 程式碼翻譯成資料庫看得懂的 SQL 指令。

然後執行 **npm run migrate**把 **SQL 檔案執行到資料庫中**

- 它做了什麼：Drizzle 會連線到真實的資料庫（PostgreSQL），把剛剛 generate 產生的 .sql 檔案拿去執行。
- 產出的結果：在資料庫中真正建立或修改資料表、欄位和關聯性。
- 重要特點：這個步驟才真正改變資料庫。執行後資料庫裡就真的有那些表可以開使存取資料了。

1. 建立資料夾 終端機輸入mkdir 資料夾名
2. 打開該資料夾的vscode
3. Npm init -y 生成package.json
4. Type後面改module
5. Npm install express
6. 新增app.js
7. 寫基礎
8. 新增腳本“dev”: “node app.js”,
9. npm run dev
10. 開postman 選GET
11. 輸入<http://localhost:3000/drinks>
12. 安裝nodemon : npm i nodemon
13. 去修改腳本 : "dev": "nodemon app.js",

第7點的基礎可以寫這樣

```
import express from "express";
```

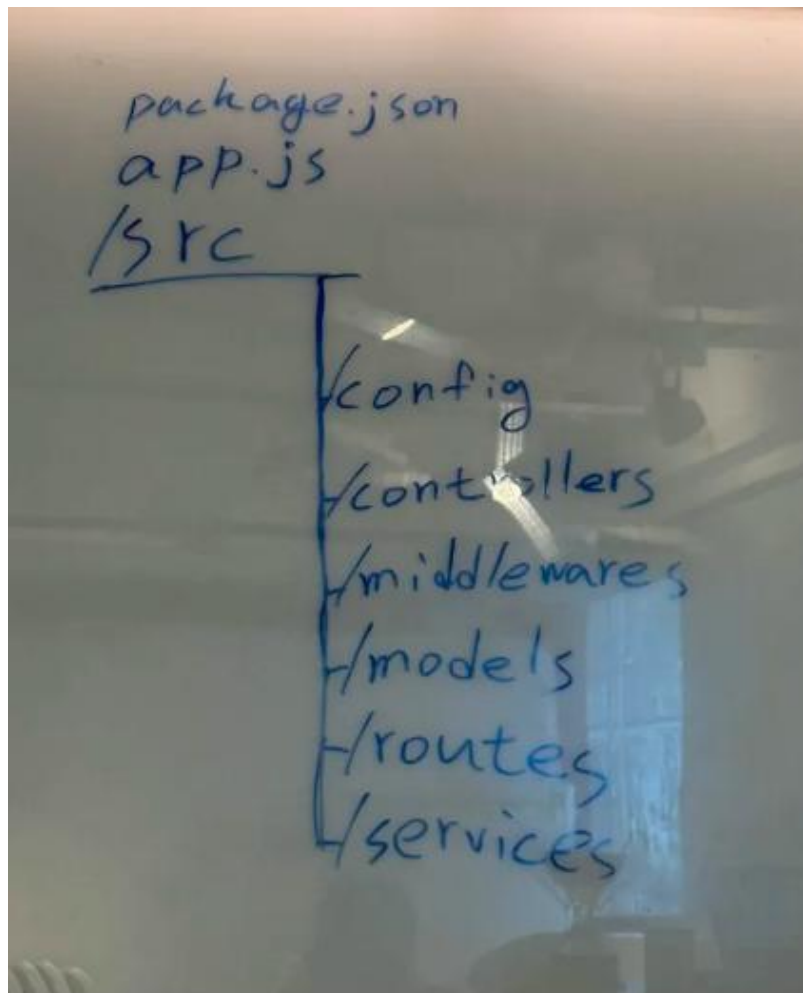
```
const app = express();
```

```
const drinks = [  
  { id: 1, name: "奶茶" },  
  { id: 2, name: "綠茶" },  
];
```

```
app.get("/drinks", (req, res) => {  
  res.json(drinks);  
});
```

```
app.listen(3000, () => {  
  console.log("啟動中");  
});
```

```
9
10 // Middleware (中間件) 指的是 logger 函式 以及 app.use(logger) 這個執行動作。在收到req和送出
    res之間，進行檢查、修改或記錄。
11 // next()的作用是告訴 Express：「中間件的工作已完成，請放行並讓請求進入下一個中間件或路由處理
    器」。
12 function logger(req, res, next) {
13   console.log(`${req.method}, ${req.url}`);
14   next();
15 }
16
17 function noFound(req, res, next) {
18   res.status(404).json({ error: "找不到網址，再試試其他的" });
19 }
20
21 // 因為 Node.js / Express 預設看不懂前端傳來的 JSON 資料，要寫app.use(express.json())才能順
    利執行。
22 app.use(express.json());
23 app.use(logger);
24
25 app.get("/drinks", (req, res) => {
26   res.json(drinks);
27 });
28
29 app.post("/drinks", (req, res) => {
30   const { id, name } = req.body;
31   drinks.push({ id, name });
32   res.status(201).json(drinks);
33 });
34
```



package.json

專案的管理文件。

記載專案資訊、套件依賴 (dependencies) 與執行腳本 (scripts) 。

app.js

專案的進入點 (Entry Point) 。

負責初始化伺服器、連接資料庫、掛載基礎中間件與啟動監聽埠。

/src

原始碼 (Source Code) 的主要目錄。

所有核心業務邏輯都會放在這個資料夾內，其下方包含以下子目錄：

- `/config`：設定ORM,etc。存放環境變數、資料庫連線或第三方服務的設定檔。
- `/controllers`：控制器。負責處理 HTTP 請求、調用業務邏輯並回傳回應。
- `/middlewares`：中間件。負責在請求到達控制器前進行攔截處理 (如：權限驗證、錯誤處理、日誌記錄) 。
- `/models`：資料庫模型。定義資料庫的 Schema (資料結構) 與資料操作方法。
- `/routes`：路由定義。負責定義 API 的端點 (Endpoints) 並將請求分流至對應的控制器。
- `/services`：資料操作。服務層。負責處理具體的業務邏輯，將複雜的運算與資料庫操作與控制器分離。

1. 建立資料夾 終端機輸入mkdir 資料夾名
2. 打開該資料夾的vscode
3. Npm init -y 生成package.json
4. Type後面改module
5. Npm install express
6. Npm install express-session
7. 新增app.js
8. 寫基礎
9. 新增腳本“dev”: “node app.js”,
10. npm run dev
11. 然後按照老師教的創很多資料夾和檔案

```
基礎
import express from "express";

const app = express();

app.listen(3000, () => {
  console.log("啟動中");
});
```